

Experiment No: 9

Title: Deployment and Configuration options in Microsoft Azure.

Aim: To study Deployment and Configuration options in Microsoft Azure.

Theory: You can use a few different technologies to deploy your Azure Functions project code to Azure. This article provides an overview of the deployment methods available to you and recommendations for the best method to use in various scenarios. It also provides an exhaustive list of and key details about the underlying deployment technologies.

Deployment methods:

The deployment technology you use to publish code to Azure is generally determined by the way in which you publish your app. The appropriate deployment method is determined by specific needs and the point in the development cycle. For example, during development and testing you may deploy directly from your development tool, such as Visual Studio Code. When your app is in production, you are more likely to publish continuously from source control or by using an automated publishing pipeline, which includes additional validation and testing.

The following table describes the available deployment methods for your Function project.

Deployment type	DEPLOYMENT METHODS	
	Methods	Best for...
Tools-based	• Visual Studio Code publish • Visual Studio publish • Core Tools publish	Deployments during development and other ad-hock deployments. Deployments are managed locally by the tooling.
App Service-managed	• Deployment Center (CI/CD) • Container deployments	Continuous deployment (CI/CD) from source control or from a container registry. Deployments are managed by the App Service platform (Kudu).
External pipelines	• Azure Pipelines • GitHub actions	Production and DevOps pipelines that include additional validation, testing, and other actions be run as part of an automated deployment. Deployments are managed by the pipeline.

Deployment technology availability

Azure Functions supports cross-platform local development and hosting on Windows and Linux. Currently, three hosting plans are available:

- Consumption
- Premium
- Dedicated (App Service)

Each plan has different behaviors. Not all deployment technologies are available for each flavor of Azure Functions.

Key concepts

Some key concepts are critical to understanding how deployments work in Azure Functions.

Trigger syncing

When you change any of your triggers, the Functions infrastructure must be aware of the changes. Synchronization happens automatically for many deployment technologies.

However, in some cases, you must manually sync your triggers. When you deploy your updates by referencing an external package URL, local Git, cloud sync, or FTP, you must manually sync your triggers. You can sync triggers in one of three ways:

- Restart your function app in the Azure portal
- Send an HTTP POST request to `https://{functionappname}.azurewebsites.net/admin/host/synctriggers?code=<API_KEY>` using the master key.
- Send an HTTP POST request to `https://management.azure.com/subscriptions/<SUBSCRIPTION_ID>/resourceGroups/<RESOURCE_GROUP_NAME>/providers/Microsoft.Web/sites/<FUNCTION_APP_NAME>/syncfunctiontriggers?api-version=2016-08-01`. Replace the placeholders with your subscription ID, resource group name, and the name of your function app.

Remote build

Azure Functions can automatically perform builds on the code it receives after zip deployments. These builds behave slightly differently depending on whether your app is running on Windows or Linux. Remote builds are not performed when an app has previously been set to run in Run from Package mode. To learn how to use remote build, navigate to zip deploy.

Note

If you're having issues with remote build, it might be because your app was created before the feature was made available (August 1, 2019). Try creating a new function app, or running `azfunctionapp update -g <RESOURCE_GROUP_NAME> -n <APP_NAME>` to update your function app. This command might take two tries to succeed.

All function apps running on Windows have a small management app, the SCM (or Kudu) site. This site handles much of the deployment and build logic for Azure Functions.

When an app is deployed to Windows, language-specific commands, like `dotnet restore` (C#) or `npm install` (JavaScript) are run.

Remote build on Linux

To enable remote build on Linux, the following application settings must be set:

- `ENABLE_ORYX_BUILD=true`
- `SCM_DO_BUILD_DURING_DEPLOYMENT=true`

By default, both Azure Functions Core Tools and the Azure Functions Extension for Visual Studio Code perform remote builds when deploying to Linux. Because of this, both tools automatically create these settings for you in Azure.

When apps are built remotely on Linux, they run from the deployment package.

Consumption plan

Linux function apps running in the Consumption plan don't have an SCM/Kudu site, which limits the deployment options. However, function apps on Linux running in the Consumption plan do support remote builds.

Dedicated and Premium plans

Function apps running on Linux in the Dedicated (App Service) plan and the Premium plan also have a limited SCM/Kudu site.

Deployment technology details

The following deployment methods are available in Azure Functions.

External package URL

You can use an external package URL to reference a remote package (.zip) file that contains your function app. The file is downloaded from the provided URL, and the app runs in Run From Package mode.

How to use it: Add `WEBSITE_RUN_FROM_PACKAGE` to your application settings. The value of this setting should be a URL (the location of the specific package file you want to run). You can add settings either in the portal or by using the Azure CLI.

If you use Azure Blob storage, use a private container with a shared access signature (SAS) to give Functions access to the package. Any time the application restarts, it fetches a copy of the content. Your reference must be valid for the lifetime of the application.

When to use it: External package URL is the only supported deployment method for Azure Functions running on Linux in the Consumption plan, if the user doesn't want a remote build to occur. When you update the package file that a function app references, you must manually sync triggers to tell Azure that your application has changed.

Zip deploy

Use zip deploy to push a .zip file that contains your function app to Azure. Optionally, you can set your app to start running from package, or specify that a remote build occurs.

How to use it: Deploy by using your favorite client tool: Visual Studio Code, Visual Studio, or from the command line using the Azure Functions Core Tools. By default, these tools use zip deployment and run from package. Core Tools and the Visual Studio Code extension both enable remote build when deploying to Linux. To manually deploy a .zip file to your function app, follow the instructions in [Deploy from a .zip file or URL](#).

When you deploy by using zip deploy, you can set your app to run from package. To run from package, set the WEBSITE_RUN_FROM_PACKAGE application setting value to 1. We recommend zip deployment. It yields faster loading times for your applications, and it's the default for VS Code, Visual Studio, and the Azure CLI.

When to use it: Zip deploy is the recommended deployment technology for Azure Functions.

Docker container

You can deploy a Linux container image that contains your function app.

How to use it: Create a Linux function app in the Premium or Dedicated plan and specify which container image to run from. You can do this in two ways:

Create a Linux function app on an Azure App Service plan in the Azure portal. For Publish, select Docker Image, and then configure the container. Enter the location where the image is hosted.

Create a Linux function app on an App Service plan by using the Azure CLI. To learn how, see [Create a function on Linux by using a custom image](#).

To deploy to an existing app by using a custom container, in Azure Functions Core Tools, use the func deploy command.

When to use it: Use the Docker container option when you need more control over the Linux environment where your function app runs. This deployment mechanism is available only for Functions running on Linux.

Source control

Use source control to connect your function app to a Git repository. An update to code in that repository triggers deployment. For more information, see the [Kudu Wiki](#).

How to use it: Use Deployment Center in the Functions area of the portal to set up publishing from source control. For more information, see [Continuous deployment for Azure Functions](#).

When to use it: Using source control is the best practice for teams that collaborate on their function apps. Source control is a good deployment option that enables more sophisticated deployment pipelines.

You can use local Git to push code from your local machine to Azure Functions by using Git.

How to use it: Follow the instructions in Local Git deployment to Azure App Service.

When to use it: In general, we recommend that you use a different deployment method. When you publish from local Git, you must manually sync triggers.

Cloud sync

Use cloud sync to sync your content from Dropbox and OneDrive to Azure Functions.

How to use it: Follow the instructions in Sync content from a cloud folder.

When to use it: In general, we recommend other deployment methods. When you publish by using cloud sync, you must manually sync triggers.

FTP

You can use FTP to directly transfer files to Azure Functions.

How to use it: Follow the instructions in Deploy content by using FTP/s.

When to use it: In general, we recommend other deployment methods. When you publish by using FTP, you must manually sync triggers.

Portal editing

In the portal-based editor, you can directly edit the files that are in your function app (essentially deploying every time you save your changes).

How to use it: To be able to edit your functions in the Azure portal, you must have created your functions in the portal. To preserve a single source of truth, using any other deployment method makes your function read-only and prevents continued portal editing. To return to a state in which you can edit your files in the Azure portal, you can manually turn the edit mode back to Read/Write and remove any deployment-related application settings (like `WEBSITE_RUN_FROM_PACKAGE`).

When to use it: The portal is a good way to get started with Azure Functions. For more intense development work, we recommend that you use one of the following client tools:

- Visual Studio Code
- Azure Functions Core Tools (command line)
- Visual Studio

Deployment behaviors

When you do a deployment, all existing executions are allowed to complete or time out, after which the new code is loaded to begin processing requests.

If you need more control over this transition, you should use deployment slots.

When you deploy your function app to Azure, you can deploy to a separate deployment slot instead of directly to production. For more information on deployment slots.

Azure App Configuration

Azure App Configuration provides a service to centrally manage application settings and feature flags. Modern programs, especially programs running in a cloud, generally have many components that are distributed in nature. Spreading configuration settings across these components can lead to hard-to-troubleshoot errors during an application deployment. Use App Configuration to store all the settings for your application and secure their accesses in one place.

Why use App Configuration?

Cloud-based applications often run on multiple virtual machines or containers in multiple regions and use multiple external services. Creating a robust and scalable application in a distributed environment presents a significant challenge.

Various programming methodologies help developers deal with the increasing complexity of building applications. For example, the Twelve-Factor App describes many well-tested architectural patterns and best practices for use with cloud applications. One key recommendation from this guide is to separate configuration from code. An application's configuration settings should be kept external to its executable and read in from its runtime environment or an external source.

While any application can make use of App Configuration, the following examples are the types of application that benefit from the use of it:

- Microservices based on Azure Kubernetes Service, Azure Service Fabric, or other containerized apps deployed in one or more geographies
- Serverless apps, which include Azure Functions or other event-driven stateless compute apps
- Continuous deployment pipeline

App Configuration offers the following benefits:

- A fully managed service that can be set up in minutes
- Flexible key representations and mappings
- Tagging with labels
- Point-in-time replay of settings
- Dedicated UI for feature flag management
- Comparison of two sets of configurations on custom-defined dimensions
- Enhanced security through Azure-managed identities
- Encryption of sensitive information at rest and in transit
- Native integration with popular frameworks

App Configuration complements Azure Key Vault, which is used to store application secrets. App Configuration makes it easier to implement the following scenarios:

- Centralize management and distribution of hierarchical configuration data for different environments and geographies
- Dynamically change application settings without the need to redeploy or restart an application
- Control feature availability in real-time

Use App Configuration

The easiest way to add an App Configuration store to your application is through a client library provided by Microsoft. The following methods are available to connect with your application, depending on your chosen language and framework

USE APP CONFIGURATION

Programming language and framework	How to connect
.NET Core and ASP.NET Core	App Configuration provider for .NET Core
.NET Framework and ASP.NET	App Configuration builder for .NET
Java Spring	App Configuration client for Spring Cloud
Others	App Configuration REST API

Conclusion: Thus, we have studied Deployment and Configuration options in Microsoft Azure.

Experiment No: 10

Title: Assignment to install and configure Google App Engine

Theory:

Before begin

1. [Create a Google Cloud Platform project](#), if you don't have one already.
2. Make sure that Python 2.7 is installed on your

system: python -V

Note: As of Cloud SDK version 206.0.0, the gcloud CLI has experimental support for running using a Python 3.4+ interpreter (run gcloud topic startup for exclusions and more information on configuring your Python interpreter). All other Cloud SDK tools still require a Python 2.7 interpreter.

3. **Download the archive file best suited to your operating system.** Most machines will run the 64-bit package. If you'd like to check, run uname -m to verify if you're running a 64-bit system.

Platform	Package	Size	SHA256 Checksum
Linux			
64-bit	google-cloud-sdk-229.0.0-linux-x86_64.tar.gz	25.6 MB	b1c87fc9451598a76cf66978dd8aa06482bfc6d639b56cf31559dc2c7f8b7b90
(x86_64)	x86_64.tar.gz		
Linu			
x 32-bit	google-cloud-sdk-229.0.0-linux-x86.tar.gz	25.2 MB	ee8c45f8018d0fee92b07c32cc6d8c891241da0b88bfe289d4e58e6746c3f668
(x86)	x86.tar.gz		

Alternatively, to download the Linux 64-bit archive file from your command-line, run:

```
curl -O https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/google-cloud-sdk-229.0.0-linux-x86_64.tar.gz
```

For the 32-bit archive file, run:

```
curl -O https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/google-
```

cloud-sdk- 229.0.0-linux-x86.tar.gz

4. Extract the archive to any location on your file system; preferably, your Home folder.

On Linux, you can extract the archive file by running this command:

```
tar zxvf [ARCHIVE_FILE] google-cloud-sdk
```

5. If you're having trouble getting the gcloud command to work, ensure your \$PATH is defined appropriately. Use the install script to add Cloud SDK tools to your path. You will also be able to opt-in to command-completion for your bash shell and [usage statistics collection](#) during the installation process. Run the script using this command:

```
./google-cloud-sdk/install.sh
```

Restart your terminal for the changes to take effect.

Alternatively, you can call Cloud SDK after extracting the downloaded archive by invoking its executables via the full path.

Initialize the SDK

Use the gcloud init command to perform several common SDK setup tasks. These include authorizing the SDK tools to access Google Cloud Platform using your user account credentials and setting up the default SDK configuration.

To initialize the SDK:

1. Run the following at a command

```
prompt: gcloud init
```

Note: To prevent the command from launching a web browser, use `gcloud init --console-` only instead. To authorize without a web browser and non-interactively, create a service account with the appropriate scopes using the [Google Cloud Platform Console](#) and use `gcloud auth activate-service-account` with the corresponding JSON key file.

2. Accept the option to log in using your Google user account:

To continue, you must log in. Would you like to log in (Y/n)? Y

3. In your browser, log in to your Google user account when prompted and click **Allow** to grant permission to access Google Cloud Platform resources.
4. At the command prompt, select a Cloud Platform project from the list of those where

Department of Computer Science and Engineering
you have **Owner**, **Editor** or **Viewer** permissions:

Pick cloud project to use:

[1] [my-project-1]

[2] [my-project-2]

...

Please enter your numeric choice:

If you only have one project, gcloud init selects it for you.

5. If you have the Google Compute Engine API enabled, gcloud init allows you to choose a default Compute Engine zone:

Which compute zone would you like to use as project default?

[1] [asia-east1-a]

[2] [asia-east1-b]

...

[14] Do not use default zone

Please enter your numeric

choice:

gcloud init confirms that you have complete the setup steps successfully: gcloud has now been configured!

You can use [gcloud config] to change more gcloud settings.

Your active configuration is: [default]

Run core gcloud commands

Run these gcloud commands to view information about your SDK installation:

1. To list accounts whose credentials are stored on the local

system: gcloud auth list

gcloud displays a list of credentialed accounts:

Credentialed Accounts

ACTIVE ACCOUNT

example-user-1@gmail.com

example-user-2@gmail.com

2. To list the properties in your active SDK configuration:

gcloud config list

gcloud displays the list of properties:

[core]

account = example-user-
1@gmail.com

disable_usage_reporting = False

project = example-project

3. To view information about your Cloud SDK installation and the active SDK configuration: gcloud info

gcloud displays a summary of information about your Cloud SDK installation. This includes information about your system, the installed SDK components, the active user account and current project, and the properties in the active SDK configuration.

4. To view information about gcloud commands and other topics from the

command line: gcloud help

For example, to view the help for gcloud compute instances create:

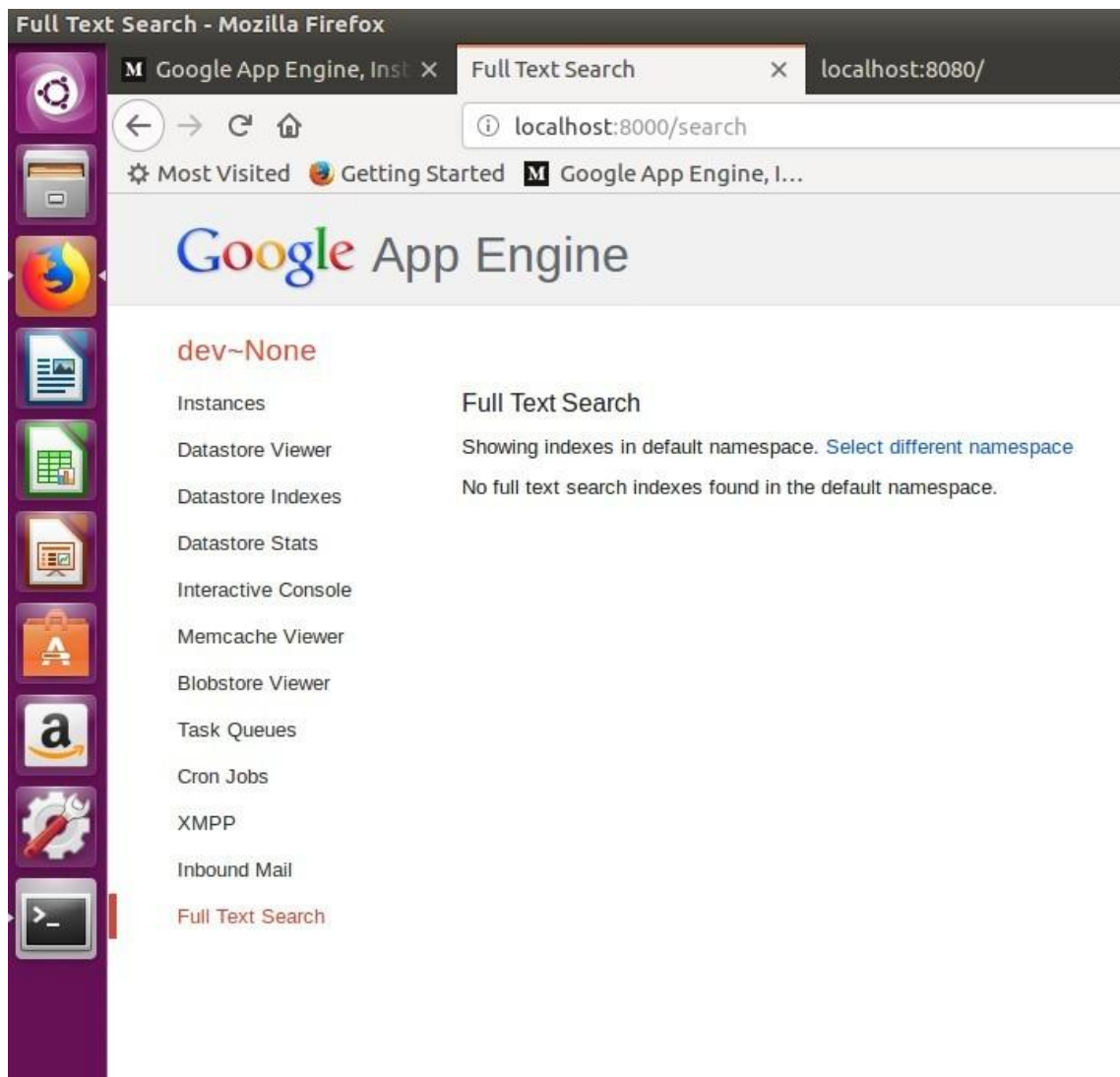
gcloud help compute instances create

gcloud displays a help topic that contains a description of the command, a list of commandflags and arguments, and examples of how to use it.

How to Run Program:

Now as we have finished installing app engine, now it's time to create and upload an app. In this case we will be taking example of a "HELLO WORLD" app in python.

1. As we already have made sure that we have python installed in our system, It will be easier for us to clone existing code and deploy it rather than creating our own so we will use python docs- sample. Run the command "git clone <https://github.com/GoogleCloudPlatform/python-docs-samples>".
2. `cd python docs- samples/appengine/standard/hello_world`
3. `dev_appserver.py app.yaml`



Conclusion: Thus, we have installed and configure Google App Engine